

CI/CD for Go (Golang) Logistics App

Course Name: DevOps

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	JIYANSHI KESHRI	EN22CS303026
2	MANASVI JAIN	EN22CS301580
3	KRATIKA TOMAR	EN22CS301516
4	MADHURI PRAJAPATI	EN22CS301562
5	NIKHIL KUMAR SINGH	EN22CS301648
6	PRIYANKA KUMARI	EN22CS301758

Group Name: Placed Btech Group 05

Project Number: DO -05

Industry Mentor Name:

University Mentor Name:

Academic Year:2026

Problem Statement & Objectives

1. Problem Statement

In today's software development process, many developers face challenges related to inconsistent environments, manual deployment errors, and lack of automation. Applications often behave differently on different systems due to dependency issues, which makes debugging and deployment difficult.

To overcome these problems, there is a need for a system that can automate building, testing, and deployment while ensuring consistency across environments. This project focuses on solving these issues by developing a Go-based logistics application integrated with a CI/CD pipeline and Docker containerization.

2. Project Objectives

The main objective of this project is to simplify and automate the development workflow. The project aims to:

- Develop a lightweight logistics web application using Go
- Implement an automated CI/CD pipeline using GitHub Actions
- Use Docker to containerize the application
- Ensure consistent performance across different environments
- Reduce manual intervention in testing and deployment

3. Scope of the Project

This project mainly focuses on backend development and DevOps practices. It includes building the application, writing test cases, setting up CI/CD, and containerizing the system.

However, it does not cover:

- Deployment on cloud platforms
- Advanced orchestration tools like Kubernetes
- Complex real-world logistics features

Proposed Solution

1.Key features

The project includes the following key features:

- Simple and efficient Go HTTP server
- Automated CI/CD pipeline
- Docker-based containerization
- Modular project structure
- Unit testing integration
- Easy setup and execution

2.Overall Architecture / Workflow

The system follows a simple but effective workflow. First, the developer writes the code and pushes it to GitHub. Once the code is pushed, the CI/CD pipeline is automatically triggered.

The pipeline performs multiple steps such as:

- Checking out the code
- Setting up the Go environment
- Running test cases
- Building the application

After this, Docker is used to build an image of the application, which is then run inside a container. The user can access the application through a browser.

In simple terms, the flow looks like this:

Developer → GitHub → CI/CD Pipeline → Build & Test → Docker → Container → User
This automated process helps in reducing errors and improving efficiency.

3.Tools & Technologies Used

This project uses a combination of development and DevOps tools to achieve automation and consistency.

The main technologies used are:

- **Go (Golang)** – Used for building the backend application
- **Git & GitHub** – Used for version control and code hosting
- **GitHub Actions** – Used to create and manage the CI/CD pipeline
- **Docker** – Used to containerize the application
- **VS Code** – Used as the development environment

From the project structure (as shown in your image), we can understand:

- `.github/workflows/ci-cd.yml` → Defines CI/CD pipeline

- cmd/main.go → Entry point of the application
- internal/handlers/ → Contains business logic
- tests/ → Contains test files
- Dockerfile → Defines container build process
- go.mod → Manages dependencies

Results & Output

Add the below details here:

1. Screenshots / outputs

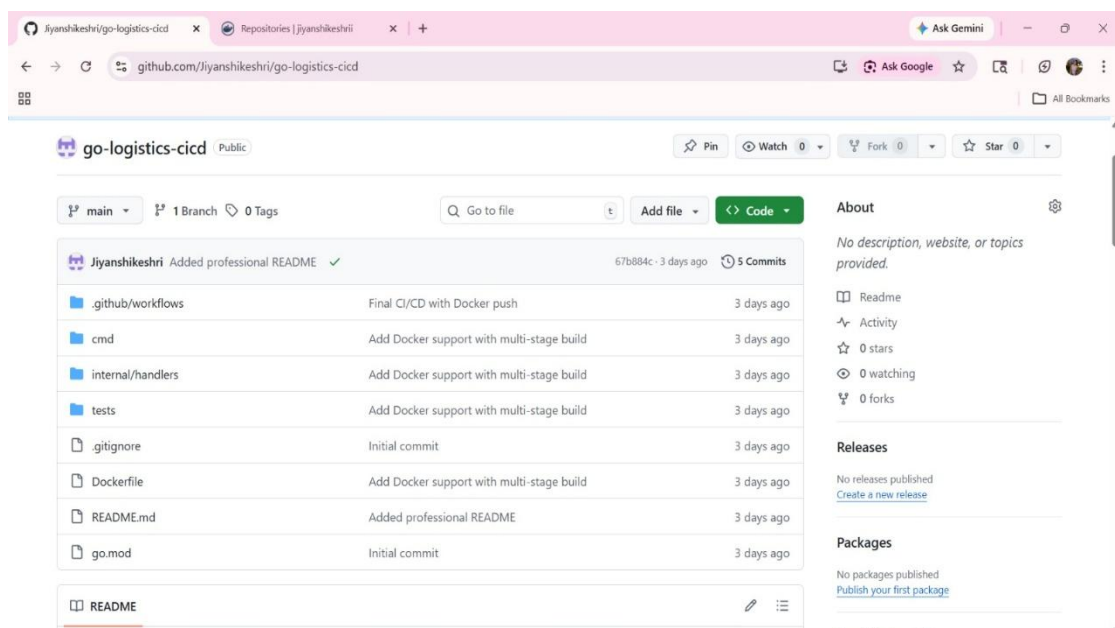


Fig.1 Project folder structure

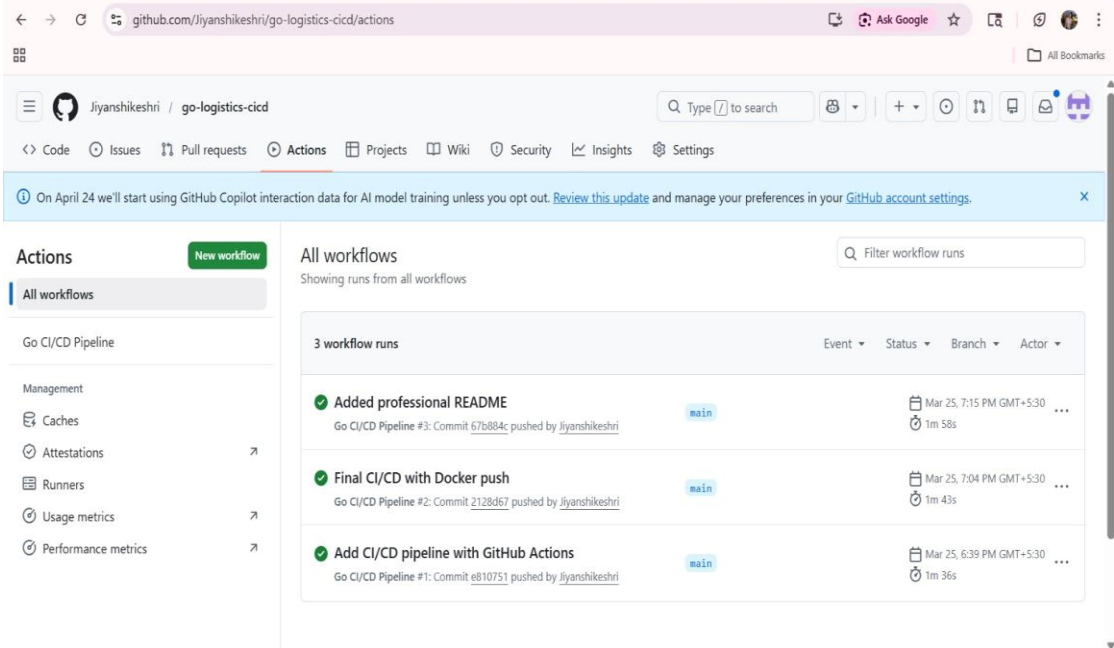


Fig.2 CI/CD pipeline running in GitHub Actions

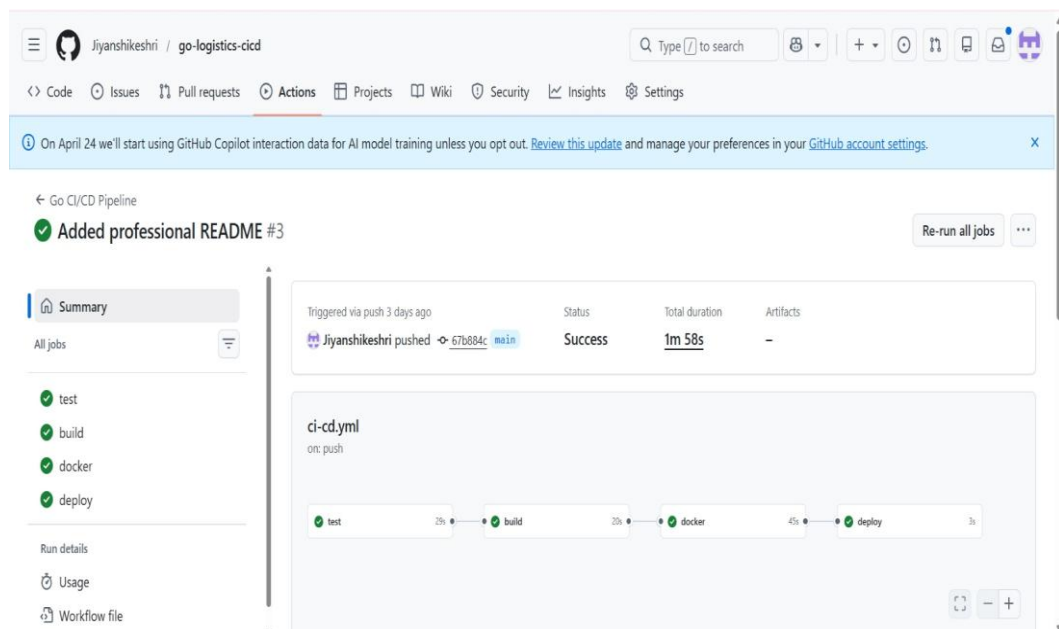


Fig.3 Added professional Readme

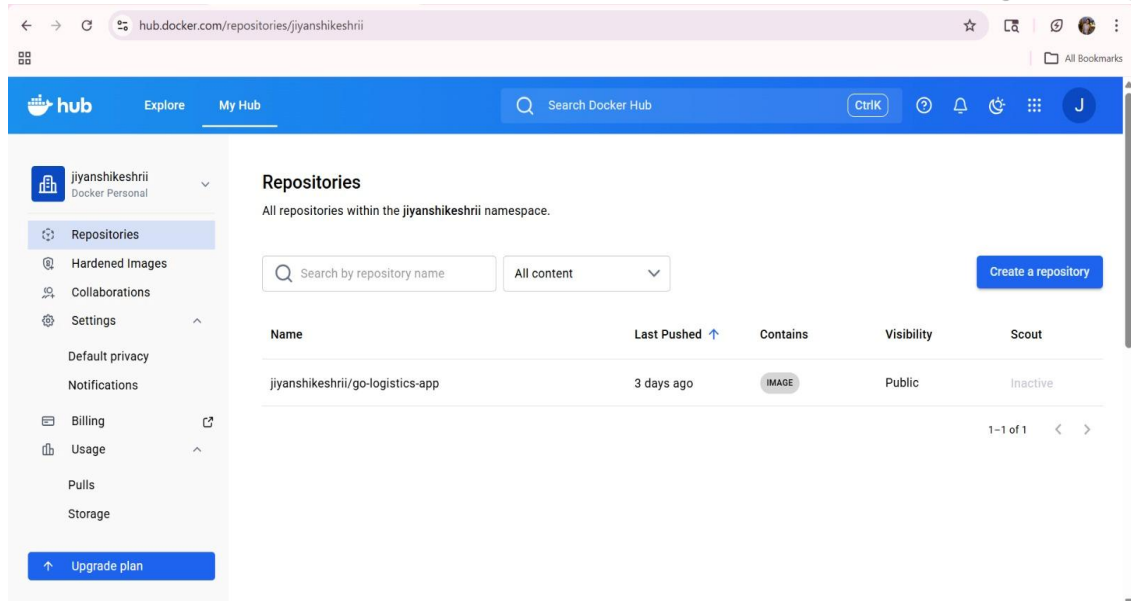


Fig.4 Docker Hub Repository

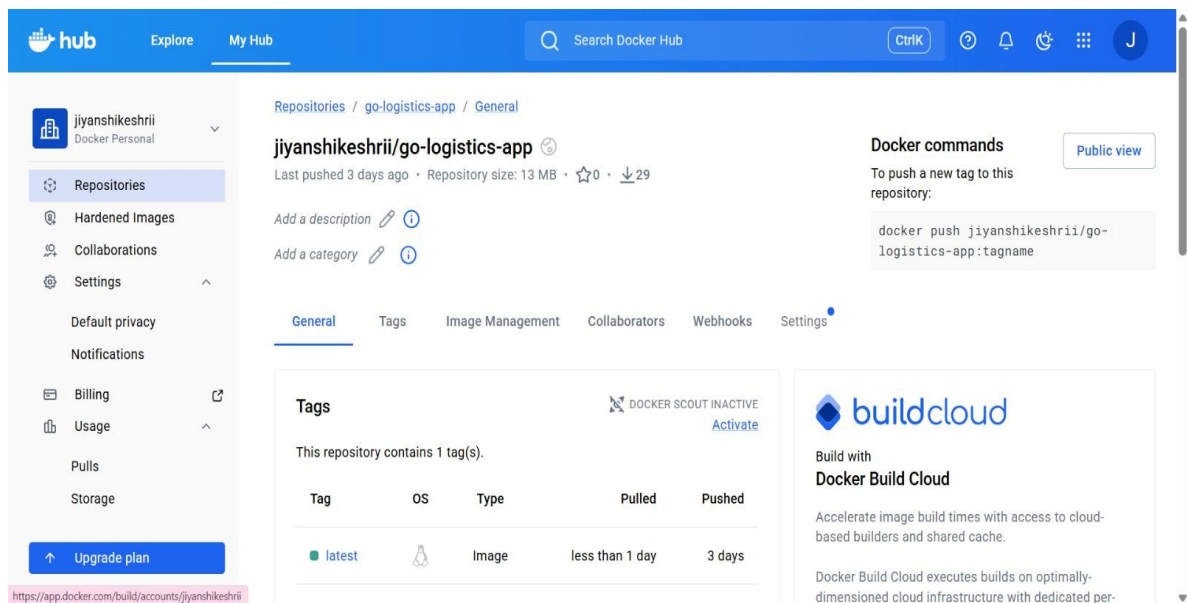


Fig.5 go-logistics-app

2. Reports / dashboards / models

The CI/CD pipeline itself acts as a report/dashboard where you can see:

- Build status (success/failure)
- Test execution results
- Workflow logs

These logs help in debugging and monitoring the application automatically.

3. Key outcomes

This project successfully demonstrates how modern DevOps practices can be applied to a simple application.

Some important outcomes include:

- Automation of build and testing process
- Successful implementation of CI/CD pipeline
- Containerized application using Docker
- Improved reliability and consistency
- Better development workflow

Conclusion

This project combines backend development with DevOps practices to create a complete working system. By integrating Go with Docker and CI/CD, the project ensures that the application is easy to build, test, and run in any environment.

Through this project, we understood the importance of automation, containerization, and structured development. It also helped in gaining practical knowledge of real-world development workflows.

Future Scope & Enhancements

Although the project works well, there are many areas where it can be improved further.

Some possible future enhancements are:

- Deploying the application on cloud platforms like AWS or Azure
- Using Kubernetes for container orchestration
- Adding database integration for real data handling
- Implementing authentication and security features
- Expanding the application with real logistics functionalities (tracking, orders, etc.)

